



Detector de Chollos

en Alojamientos Turísticos de Andalucía

Detección de anomalías negativas en precios mediante ML supervisado (regresión) y categorización automática de chollos · App web Streamlit

Mario Jiménez

Trabajo de Fin de Grado

Universidad de Sevilla · 2025-2026

Python 3.12

R

Selenium

Playwright

XGBoost

Streamlit

Scikit-learn

Mapa de Aplicación de Contenidos

Distribución de herramientas

Tema	Herramientas	Archivo / Función	Función en el proyecto
Web Scraping (Fase 1)	Selenium, webdriver-manager, BeautifulSoup4	Scraping/Scrp_estancias_provincias.py → scrape_con_subdivision(), cargar_todo()	Listados Booking: título, URL, valoración, tipo, estrellas
Web Scraping (Fase 2)	Playwright, requests, GraphQL API Booking	Scraping/Scrp_caracteristicas_estancias.py → BookingSession, _amenities(), _calendar()	Fichas: ubicación GPS, servicios, 365 días precios
Preprocesamiento	pandas, numpy, ast, json, regex	Cleaning/preprocessing.py → extraer_servicios(), haversine_km_vec()	Servicios binarios, distancia Haversine, feature eng.
Regresión (ML sup.)	sklearn: LR, DT, RF · xgboost	Modelizacion/Modelizacion.py → crear_bosque_aleatorio(), crear_boosting()...	Predicción del precio justo de referencia
Categorización Chollo	numpy.np.select · pandas.Series	Modelizacion/Modelizacion.py → crear_etiqueta_chollo(y_real, y_predicho)	5 categorías: inflado/normal/chollo/super/hiper (ratio)
Validación y Tuning	GridSearchCV, K-Fold cv=3, train_test_split r2_score, MAE, RMSE	Modelizacion/Modelizacion.py → train_test_validation_particion()	Holdout 70/15/15 · X_test tocado una sola vez
Visualización	seaborn, matplotlib, plotly.express, plot_tree	Graficos/Grafico_Alojamientos_Andalucia.py Modelizacion/Modelizacion.py	Mapa coroplético, correlación, árbol, scatter
App web + serialización	Streamlit, joblib	App/main.py · App/pages/Busqueda.py	UI: formulario → scraping → regresión → chollos
Gestión entorno	uv, pyproject.toml, python-dotenv	pyproject.toml · utils.py	Entorno reproducible · paquetes editables · lockfile

Gestión del Entorno y Arquitectura

uv · pyproject.toml · python-dotenv · paquetes editables

pyproject.toml

```
# pyproject.toml (dependencias reales)
[project]
name = "TFG"
dependencies = [
    "beautifulsoup4>=4.14.3",
    "ipykernel>=7.2.0",
    "joblib>=1.4.0",
    "lxml>=6.0.2",
    "matplotlib>=3.9.0",
    "nbformat>=4.2.0",
    "numpy>=2.0.0",
    "pandas>=3.0.1",
    "playwright>=1.58.0",
    "plotly>=6.6.0",
    "python-dotenv>=1.0.0",
    "requests>=2.32.5",
    "scikit-learn>=1.8.0",
    "seaborn>=0.13.2",
    "selenium>=4.41.0",
    "setuptools>=82.0.1",
    "streamlit>=1.35",
    "webdriver-manager>=4.0.2",
    "xgboost>=3.2.0"]
```

python-dotenv + utils.py

```
# .env – rutas sin hardcode
BASE=C:/ruta/TFG/TFG_Chollos

# utils.py
def conseguir_ruta_general_TFG():
    load_dotenv()
    base_env = os.getenv("BASE")
    if base_env:
        return Path(base_env)

# Paquete editable (pip install -e .)
# → importar desde cualquier módulo
from TFG_Chollos.utils import (
    configurar_logger,
    conseguir_ruta_general_TFG)

# Logger centralizado
logging.basicConfig(
    format="%(asctime)s | %(levelname)-8s"
    " | %(message)s")
```

python-dotenv para rutas

Evita rutas absolutas hardcodeadas.

uv como gestor de dependencias

Alternativa moderna a pip. Más rápido, con uv.lock reproducible. uv sync instala todo en una orden.

Paquetes locales editables

uv pip install -e . vincula los módulos como librería. Cambios en código se reflejan sin reinstalar.

Estructura del Proyecto

TFG_Chollos/ · data/ · pyproject.toml — arquitectura modular por responsabilidad

 Scraping/	 Cleaning/	 Modelizacion/	 App/	 data/
Generador_urls_generales.py Scrp_estancias_provincias.py # Selenium · listados Scrp_caracteristicas_estancias.py # Playwright · fichas detalle obtener_coordenadas_centros.py # Geocodificación Nominatim patch_room_size.py # ThreadPoolExecutor # (5 workers)	preprocessing.py # Limpieza y estructuración post_analisis.py # Eliminación de outliers # IQR · Tukey encoding.py # Codificación para ML # LabelEnc · OHE · joblib	Modelizacion.py # Entrenamiento y evaluación # LR · DT · RF · XGBoost transformaciones.py # Partición 70 / 15 / 15 # StandardScaler Graficos/ Grafico_Alojamientos.py correlacion.R # Pearson · R 4.x · arrow	main.py # Página principal (mapa) run.py # Lanzador graficos_analisis.py _predictor.py # Predicción (joblib) _scraper_app.py # Scraping en real-time _feature_engineering.py # Preprocesado predicción pages/ Busqueda.py # Búsqueda de chollos	raw/ # Datos brutos del # scraping (CSV) processed/ # Limpios + codificados # (.parquet) models/ # .pkl entrenados resultados/ # Métricas evaluación

pyproject.toml · uv · dependencias reproducibles · pip install -e ·

utils.py · conseguir_ruta_general_TFG() · configurar_logger() · python-dotenv

Certificados de Formación

DataCamp (5 cursos · pandas, Python) + Dagster Essentials

Dagster Essentials · Dagster

Emitido: 2026-04-23 · ID: xqym2ebkbt



STATEMENT OF ACCOMPLISHMENT

HAS BEEN AWARDED TO
mariostjl202
FOR SUCCESSFULLY COMPLETING
Joining Data with pandas

LENGTH
4 HRS
COMPLETED ON
APR 28, 2026
datacamp



Joining Data with pandas

28 Apr 2026 · 4h



STATEMENT OF ACCOMPLISHMENT

HAS BEEN AWARDED TO
mariostjl202
FOR SUCCESSFULLY COMPLETING
Writing Efficient Code with pandas

LENGTH
4 HRS
COMPLETED ON
MAY 02, 2026
datacamp



Writing Efficient Code with pandas

02 May 2026 · 4h



STATEMENT OF ACCOMPLISHMENT

HAS BEEN AWARDED TO
mariostjl202
FOR SUCCESSFULLY COMPLETING
Python for R Users

LENGTH
5 HRS
COMPLETED ON
APR 30, 2026
datacamp



Python for R Users

30 Apr 2026 · 5h



STATEMENT OF ACCOMPLISHMENT

HAS BEEN AWARDED TO
mariostjl202
FOR SUCCESSFULLY COMPLETING
Reshaping Data with pandas

LENGTH
4 HRS
COMPLETED ON
MAY 06, 2026
datacamp



Reshaping Data with pandas

06 May 2026 · 4h



STATEMENT OF ACCOMPLISHMENT

HAS BEEN AWARDED TO
mariostjl202
FOR SUCCESSFULLY COMPLETING
Data Manipulation with pandas

LENGTH
4 HRS
COMPLETED ON
MAY 07, 2026
datacamp



Data Manipulation with pandas

DataCamp · 4 HRS

Completado: 07 May 2026

mariostjl202

Contexto del Problema

¿Por qué este proyecto?

8 provincias

Toda Andalucía

Booking.com

Fuente de datos

5 categorías

hiper/super/chollo/normal/inflado

ML Regresión

Precio justo → etiqueta chollo



El problema

Los precios en Booking.com varían enormemente según servicios, ubicación, temporada y demanda. Sin apoyo de ML es imposible saber si un precio es justo o es un chollo real. Los usuarios pierden oportunidades de ahorro por falta de información comparativa.



La solución

Predecir el precio justo mediante regresión ML. Comparar el precio real con el predicho y categorizar automáticamente (hiper-chollo / chollo / normal / inflado). El usuario accede a los chollos a través de una App en Streamlit con sus preferencias.

Scraping
Booking.com

Limpieza y
Preprocesamiento

Regresión ML
(precio justo)

Etiqueta
Chollo
(ratio real/pred)

App Streamlit
usuario final

Flujo completo de la App:

Usuario

Formulario
preferencias

Búsqueda
en Booking

Extrae datos
alojamientos

Aplica modelo
regresión

Categoriza
chollo

Muestra
chollos

Web Scrapping · 2 Fases Diferenciadas

Scraping/Scrp_estancias_provincias.py + Scrp_caracteristicas_estancias.py

FASE 1 — Listados por provincia (Selenium)

Scrp_estancias_provincias.py

```
# Anti-detección Selenium
options.add_argument(
    "--disable-blink-features="
    "AutomationControlled")
options.add_experimental_option(
    "excludeSwitches", ["enable-automation"])
# Bloquea imágenes y fuentes (rendimiento)
prefs = {"profile.managed_default_content_
    "settings.images": 2, ".fonts": 2}

# Superar límite 800: subdivisión recursiva
def scrape_con_subdivision(driver, clave,
    url, precio_min=0, precio_max=6000):
    url_f = filtro_precio(url, precio_min, max)
    total = n_alojamientos(driver)
    if total <= 750:
        cargar_todo(driver)
        return extraer_alojamientos(driver, clave)
    mid = (precio_min + precio_max) // 2
    return (subdivision(..., mid) +
```

Datos extraídos (Fase 1)

lugar · título · url_estancia · valoracion_clientes · n_valoraciones · tipo · estrellas ·
fecha_extraccion_listado

FASE 2 — Fichas individuales (Playwright + GraphQL API)

Scrp_caracteristicas_estancias.py

```
# BookingSession: resuelve WAF una sola vez
# luego reutiliza cookies en requests HTTP
class BookingSession:
    def _init_session(self):
        with sync_playwright() as pw:
            page.goto("https://www.booking.com/")
            cookies = page.context.cookies()
            for c in cookies:
                self.session.cookies.set(...)

# API GraphQL interna de Booking
GRAPHQL_URL = "https://www.booking.com/"
    "dml/graphql"

# FACILITIES_QUERY → servicios generales
# CALENDAR_QUERY → 365 días de precios
# Habitación más barata: Playwright reutiliza
# cookies → tr.hprr-table-cheapest-block
```

Datos extraídos (Fase 2)

hotel_id · ubicación GPS · servicios generales · servicios habitación más barata ·
calendario 365 días

Fase 2b — patch_room_size.py: extrae tamaño real de habitación ("44 m²"). Reanudable, ThreadPoolExecutor (5 workers paralelos).

Preprocesamiento de Datos

Cleaning/preprocessing.py → post_analisis.py → encoding.py

Extracción servicios + precios

```
# Extracción servicios → booleanos
servicios = [serv.lower() for serv in
ast.literal_eval(fila['servicios'])]
# → 12 columnas binarias int8:
#   Parking, Parking_gratis, Gimnasio,
#   Restaurante, Piscina, Piscina_interior,
#   Piscina_infinita, Aire, Calefaccion,
#   Vistas, Terraza, Baño_privado
```

limpiar_db_final()

```
# Limpieza y tipado final
# Feature engineering temporal
db['es_finde'] = db['fecha_disponible'].dt.dayofweek
               .isin([4,5]).astype('int8')
db['es_domingo'] = (db['fecha_disponible'].dt.dayofweek
               == 6).astype('int8')

# Localidad extraída con regex
localidad = raw['direccion'].str.extract(
    r',s*d{5}s+([^\,]+)',')[0]
```

haversine_km_vec()

```
# Distancia al centro (Haversine vectorizado)
def haversine_km_vec(lats,lons,lat2,lon2):
    R = 6371.0
    dlat = np.radians(lat2 - lats)
    dlon = np.radians(lon2 - lons)
    a = (np.sin(dlat/2)**2 +
         np.cos(np.radians(lats)) *
         math.cos(math.radians(lat2)) *
         np.sin(dlon/2)**2)
    return (R*2*np.arcsin(np.sqrt(a))).round(3)
```

```
# Coordenadas centros cargadas desde CSV
coords = cargar_coords_centros(BASE)
df = añadir_distancia_centro(df, coords)
```

Variables finales del dataset

provincia · localidad	category
latitud · longitud	float64
distancia_centro_km	float32
tipo · estrellas	cat / int8
valoracion_clientes	float32
n_valoraciones	int64
tamaño_habitacion	int16 (m²)
12 servicios binarios	int8 (0/1)
es_finde · es_domingo	int8
dias_restantes	int16
precio (target)	int32

Preprocesamiento • Gestión de Valores Nulos

Cleaning/preprocessing.py → limpiar_db_final()

✗ Eliminación de registros (dropna)

Variables obligatorias (2,2% registros):

latitud, longitud,
latitud_centro, longitud_centro,
distancia_centro_km

Sin coordenadas GPS no se puede calcular distancia_centro_km, variable clave del modelo. Pérdida asumible (2,2%).

Localidad ≠ '9':

Error de geocodificación detectado manualmente.

```
cols_obligatorias = [  
    'latitud', 'longitud',  
    'latitud_centro', 'longitud_centro',  
    'distancia_centro_km']  
db = db.dropna(subset=cols_obligatorias)
```

```
nuevos_valores = {  
    'codigo_postal': -1,  
    'tipo': 'Otro',  
    'estrellas': 1,  
    'n_valoraciones': 0,  
    'valoracion_clientes': media,  
    'tamaño_habitacion': mediana}  
db = db.fillna(nuevos_valores)
```

✓ Reemplazo de nulos (fillna)

Variable	Valor imputado	Justificación
codigo_postal	-1	Valor desconocido, no se usa en modelo
tipo	'Otro'	Sin tipo definido → categoría residual
estrellas	1	Sin clasificación → mínimo (no hotel)
n_valoraciones	0	Sin reseñas todavía → nuevo alojamiento
valoracion_clientes	media	Evita sesgo hacia extremos; robusta ante outliers
tamaño_habitacion	mediana	Más robusta que la media ante suites con m² extremos

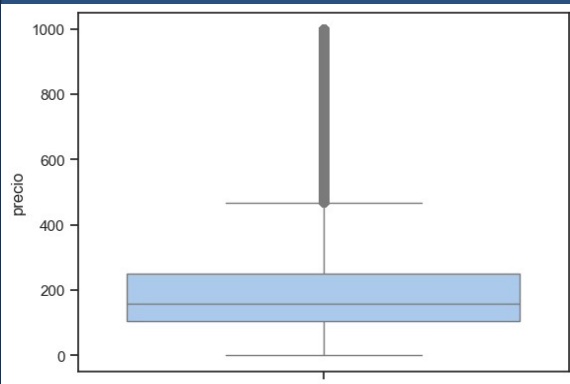
💡 Criterio general

Eliminar cuando la variable es crítica e irre recuperable. Imputar con estadístico robusto cuando la pérdida penalizaría al modelo.

Preprocesamiento · Análisis (Gestión de Outliers)

Criterio Tukey (IQR) · Cleaning/post_analisis.py · transformar_post_analisis()

💰 precio

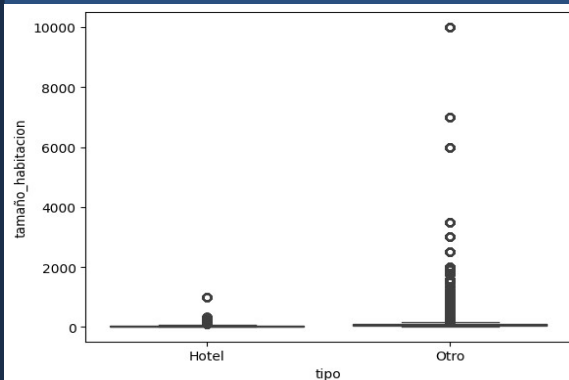


📋 Decisión:

Registros +500€/noche, raramente van a ser chollos, además distorsionarían el modelo al introducir rangos de precio tan elevados.

✗ Umbral Tukey: precio < 468€

🏠 tamaño_habitacion



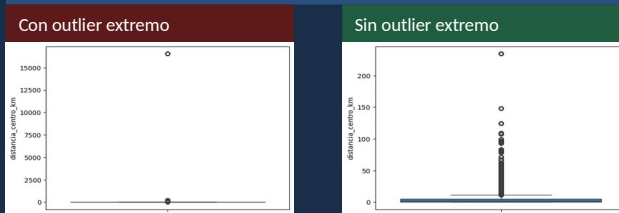
📋 Decisión:

Valores > 500 m² son fincas/villas, fuera del objeto de estudio. Hoteles = 8% del dataset.

✗ Umbral fijo: tamaño_habitacion < 500 m².

Superior al umbral Tukey para no perder casas rurales legítimas.

📍 distancia_centro_km



📋 Decisión (2 pasos):

✗ 1. Eliminar > 15.000 km:

Error GPS: alojamiento geocodificado en Islas Salomón, con dirección manual de Guadalcanal, España.

✗ 2. Umbral definitivo ≤ 15 km:

Alojamiento a >15 km del centro → probable error de geocodificación o finca rural fuera del objeto de estudio.

```
# Cleaning/post_analisis.py →  
transformar_post_analisis()  
df = df[df['distancia_centro_km'] <= 15]
```

Preprocesamiento • Encoding

Cleaning/encoding.py → encoding() • genera db_final_codificada.parquet



1. Eliminación de columnas no predictivas

Se eliminan variables identificativas que no aportan valor predictivo al modelo:

```
db_final.drop(columns=['titulo', 'codigo_postal',  
                      'url_estancia', 'fecha_extraccion'])
```



2. One-Hot Encoding — tipo

Crea columna binaria **tipo_Otro** (0=Hotel, 1=Otro).

drop_first=True elimina **tipo_Hotel** por redundancia (colinealidad perfecta).

```
db = pd.get_dummies(db, columns=['tipo'],  
                    drop_first=True)
```



3. Label Encoding — localidad + provincia

Asigna un entero único a cada categoría.

```
le_localidad = LabelEncoder()  
le_provincia = LabelEncoder()  
db['localidad'] = le_localidad.fit_transform(db['localidad'])  
db['provincia'] = le_provincia.fit_transform(db['provincia'])
```



4. Descomposición de fecha_disponible

Extrae mes y día como variables numéricas y luego elimina la columna original.

```
db['mes_disponible'] = db['fecha_disponible'].dt.month  
db['dia_disponible'] = db['fecha_disponible'].dt.day  
db = db.drop(columns=['fecha_disponible'])
```



Variables: antes → después del encoding

Variable original → Resultado [Técnica]

titulo, codigo_postal,
url_estancia, fecha_extraccion → **eliminadas** [Drop]

tipo (Hotel / Otro) → **tipo_Otro (0/1)** [OHE]

localidad (~500 valores) → **entero 0..N** [Label Enc.]

provincia (8 valores) → **entero 0..7** [Label Enc.]

fecha_disponible → **mes_disponible + dia_disponible** [Descomp.]

estrellas, n_valoraciones,
tamaño_habitacion, precio... → **ya numéricas (int8/32)** [Nativa]



Persistencia con joblib (reproducibilidad en producción)

Los encoders y la lista de columnas se guardan para replicar exactamente el encoding en producción (app Streamlit) con los mismos índices:

```
joblib.dump(le_localidad, models_dir / 'le_localidad.pkl')  
joblib.dump(le_provincia, models_dir / 'le_provincia.pkl')  
joblib.dump(columnas_modelo, models_dir / 'columnas_modelo.pkl')
```

Eficiencia del almacenamiento • RAW vs Procesado

CSV raw (por provincia) • Parquet procesado (8 provincias + todas las fechas)

Datos RAW — resultados_booking_Málaga.csv

8.871 registros • 25 columnas • Peso en disco: 279.7 MB (mostrando 14 de 25 columnas, 5 primeras filas)

lugar	titulo	tipo	estrellas	valoracion clientes	n valoraciones	fecha entrada	fecha salida	n adultos	n habitaciones	ciudad	latitud	longitud	servicios
Málaga	Ona Las Rampas	Hotel	3.0	8,4	4042.0	2026-09-16	2026-09-19	2	1	Pintor Nogales, s/n	36.54083785963548	-4.622446596622467	["Accesible para perso
Málaga	First Flatotel Interna	Hotel	3.0	7,7	4006.0	2026-09-16	2026-09-19	2	1	Ronda del Golf Oeste,	36.58162076182807	-4.550201296806336	["Acceso a pisos super
Málaga	Hch Hotel Torremolinos	Hotel	2.0	7,7	3497.0	2026-09-16	2026-09-19	2	1	Plaza de la Gamba Aleg	36.62440363477449	-4.499199360553803	["Aire acondicionado",
Málaga	Estudio céntrico en To	Apartamento/Casa/Estud	3.0	9,2	12.0	2026-09-16	2026-09-19	2	1	Avenida Isabel Manoja,	36.6231292	-4.5009589	["Acceso a pisos super
Málaga	APARTAMENTOS Los PEC	Apartamento/Casa/Estud	4.0	9,5	86.0	2026-09-16	2026-09-19	2	1	Calle La Victoria, 20	36.714167269941	-4.309819475062	["Accesible para perso



RAW —
resultados_booking_Málaga.csv
(1 provincia)

8.871 registros • 25 columnas •
279,7 MB en disco

Datos PROCESSED — db_final_analisis.parquet

2,766,237 registros • 33 columnas • Peso en disco: 11.2 MB (mostrando 17 de 33 columnas, 5 primeras filas)

provincia	localidad	tipo	estrellas	valoracion clientes	n valoraciones	distancia centro km	Piscina	Parking	Gimnasio	Restaurante	Aire	tamaño habitacion	fecha disponible	días restantes	es finde	precio
Huelva	Huelva	Hotel	3	8.300000190734863	4033	0.4539999961853027	0	0	0	0	1	15	2026-05-16 00:00:00	2	1	108
Huelva	Huelva	Hotel	3	8.300000190734863	4033	0.4539999961853027	0	0	0	0	1	15	2026-05-17 00:00:00	3	0	50
Huelva	Huelva	Hotel	3	8.300000190734863	4033	0.4539999961853027	0	0	0	0	1	15	2026-05-18 00:00:00	4	0	72
Huelva	Huelva	Hotel	3	8.300000190734863	4033	0.4539999961853027	0	0	0	0	1	15	2026-05-19 00:00:00	5	0	63
Huelva	Huelva	Hotel	3	8.300000190734863	4033	0.4539999961853027	0	0	0	0	1	15	2026-05-20 00:00:00	6	0	63



PROCESSED —
db_final_analisis.parquet (8
provincias)

2.766.237 registros • 33 columnas •
11.2 MB en disco

x312

más filas en el parquet

x25 menos

peso en disco

+8 col.

nuevas variables (features)



Por qué es importante:

El parquet tiene 312× más filas que el CSV de Málaga (combina todas las provincias y fechas), pero pesa 25× menos. Esto ilustra la eficiencia del formato Parquet: compresión columnar, tipado óptimo (int8, float32, category) y la limpieza realizada durante el preprocesamiento.

ML Supervisado • transformaciones.py

Modelizacion/transformaciones.py · partición + estandarización · módulo separado del entrenamiento

train_test_validation_particion()

Divide el dataset en 3 conjuntos con proporción 70 / 15 / 15:

Estrategia en 2 pasos:

1. Partición 70-30 → train y temporal
2. Partición 50-50 del temporal → val y test

```
X_train, X_temp, y_train, y_temp = train_test_split(
    features, target, train_size=0.7, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, train_size=0.5, random_state=42)
```

estandarizar_datos() — solo para Regresión Lineal

Aplica Z-score scaling: media = 0, desviación = 1.

Fórmula: $z = (x - \mu) / \sigma$

fit() solo sobre train:

El scaler aprende μ y σ en train. Val y test solo usan transform().
Evita filtrar información del futuro al modelo (data leakage).

```
scaler_X = StandardScaler()
X_train = scaler_X.fit_transform(X_train)
X_val = scaler_X.transform(X_val) # solo transforma
X_test = scaler_X.transform(X_test) # solo transforma
```

```
# Guardado si el mejor modelo es Regresión Lineal:
joblib.dump(scaler_X, 'scaler_X_regresion.pkl')
joblib.dump(scaler_y, 'scaler_y_regresion.pkl')
```

Diagrama de partición 70 / 15 / 15

X_train: 70%

GridSearchCV
+ K-Fold cv=3
Ajuste hiperparáms

→ Ajusta y entrena

X_val: 15%

Comparación
modelos
→ selección

→ Tomas decisiones

X_test: 15%

Evaluación
final
(solo 1 vez)

→ No tomas decisiones

¿Por qué módulo separado?

Separar transformaciones.py de Modelizacion.py permite:

- ✓ Reutilizar la partición en diferentes experimentos
- ✓ La app (_feature_engineering.py) importa solo las transformaciones sin cargar todo el pipeline ML
- ✓ Tests unitarios más fáciles de escribir
- ✓ Sigue el principio de responsabilidad única (SRP)

Uso en Modelizacion.py

```
from TFG_Chollos.Modelizacion.transformaciones import (
    train_test_validation_particion,
    estandarizar_datos,
)

# Y en la app (_feature_engineering.py):
from TFG_Chollos.Cleaning.preprocessing import (
    extraer_servicios_influyentes, añadir_distancia_centro)
```

ML Supervisado • Regresión de Precio

Modelizacion/Modelizacion.py + Modelizacion/transformaciones.py → predicción del precio justo

X_train (70%)

GridSearchCV + K-Fold cv=3
Ajuste hiperparámetros
→ mejor combinación

X_val (15%)

Comparación de modelos
y selección del mejor
→ por R^2

X_test (15%)

Evaluación final única
 R^2 , MAE, RMSE
(no se toca antes)

XGBoost

```
# Ejemplo: XGBoost
Regressor
boosting =
XGBRegressor(
    random_state=42,
    n_jobs=-1, #
    paralela interna
    verbosity=0) #
silencia output
propio
param_grid = {
    'n_estimators':
    [100, 300, 500],
    'learning_rate':
    [0.01, 0.05, 0.1],
    'max_depth':
    [3, 5],
}
grid = GridSearchCV(

estimator=boosting,

param_grid=param_grid,
cv=3, # K-Fold
sobre X_train

scoring='neg_mean_squared_error',
n_jobs=-1)
grid.fit(X_train,
y_train)
mejor =
grid.best_estimator_
```

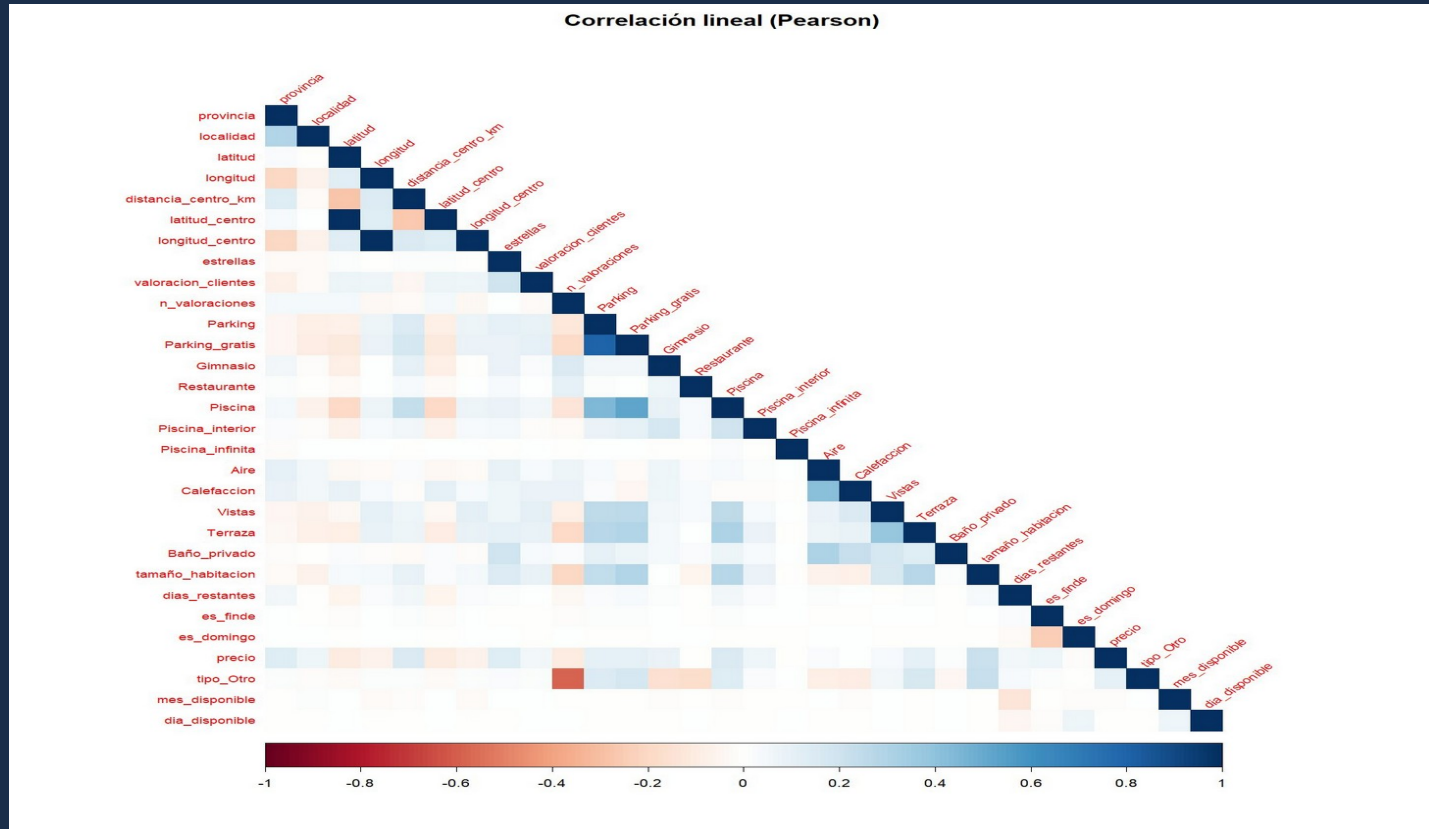
Modelo	Escalado requerido	Hiperparámetros GridSearch
Regresión Lineal	StandardScaler	—
Árbol de Decisión	Sin escalar	max_depth [3,5,10,15]
Random Forest	Sin escalar	n_estimators [50,100] max_depth [10,20]
XGBoost	Sin escalar	n_est [100,300,500] lr [0.01,0.05,0.1] max_depth [3,5]

fit_transform() solo en X_train • *transform()* en val/test → no data leakage • X_test usado una sola vez

Visualización • Corrplot de R Studio (1/3)

Correlación lineal (Pearson) • dataset codificado numéricamente

 Pearson • R Studio **corrplot** • 30 variables • LabelEncoder + get_dummies

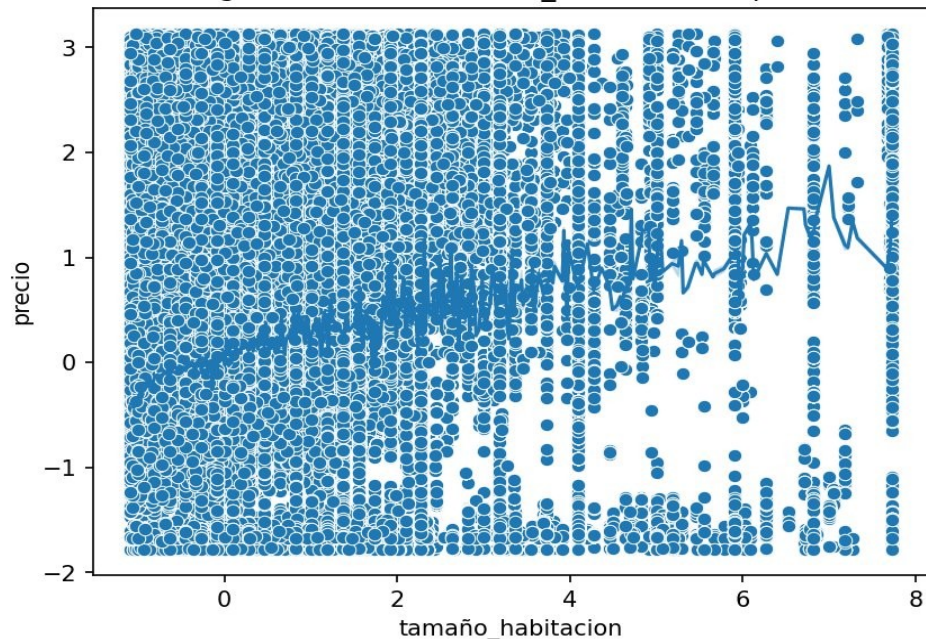


Visualización de Datos (2/3)

Regresión Lineal · Mapa Coroplético Andalucía

Regresión Lineal: tamaño_habitacion vs precio

Regresión Lineal: tamaño_habitacion vs precio



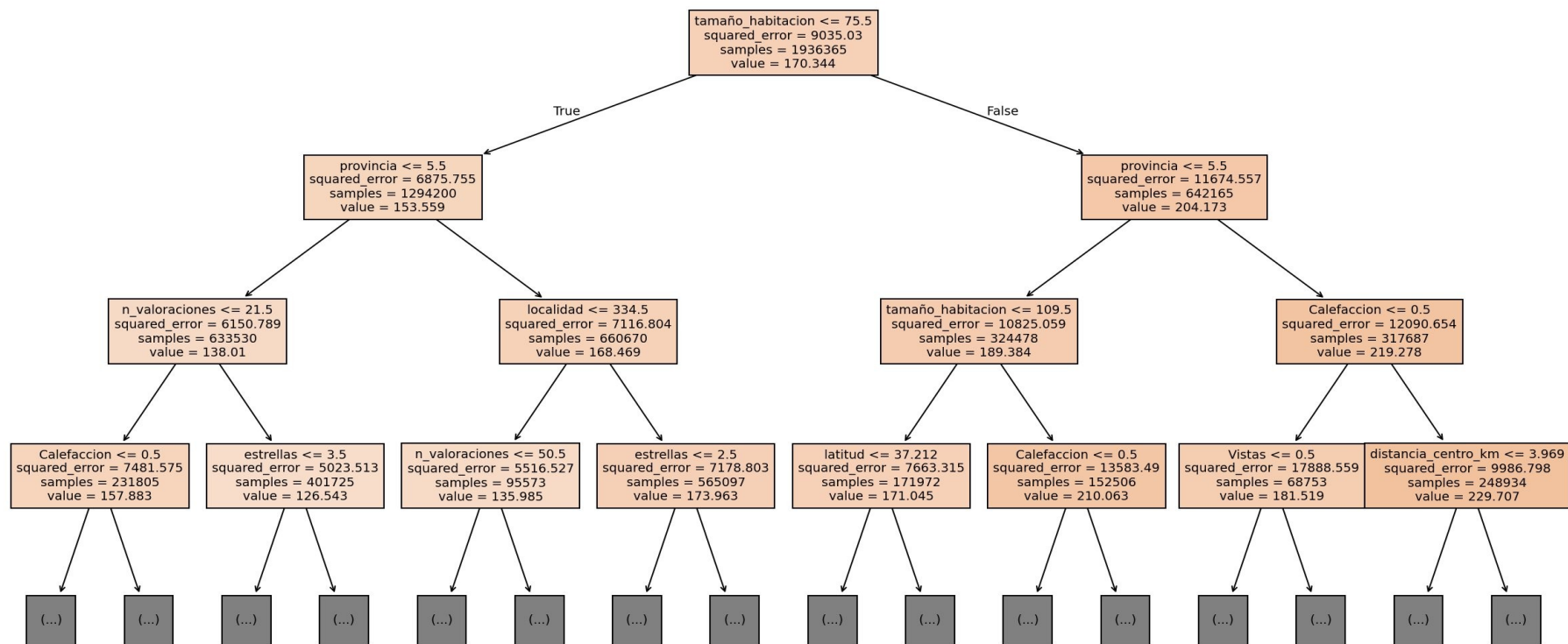
 Mapa Coroplético Andalucía · [plotly.express](#)

Choropleth por provincias andaluzas (GeoJSON spain-provinces) + scatter de alojamientos únicos (lat/lon). El HTML resultante se usa como mapa por defecto en la app Streamlit del usuario final.

```
# Mapa coroplético + scatter
fig = px.choropleth(
    locations=NOMBRES_PROVINCIAS,
    geojson=geojson,
    featureidkey="properties.name",
    color=alojamientos_por_provincia,
    color_continuous_scale="Reds")
fig.add_scattergeo(
    lat=df['latitud'],
    lon=df['longitud'],
    mode="markers",
    marker=dict(size=5,color="green"))
fig.write_html(salida) # → Streamlit
```


Visualización · Árbol de Decisión (3/3)

sklearn plot_tree · variable raíz: tamaño_habitacion $\leq$ 75.5 · primeros 4 niveles



Resultados de los Modelos • Conjunto de TEST

Modelizacion/Modelizacion.py • Evaluación final (X_test, usado una única vez)



Comparativa de modelos — Métricas sobre conjunto de TEST

Comparativa de modelos — Métricas sobre conjunto de TEST

Modelo	R ²	MAE (€)	RMSE (€)
Regresión Lineal	0.1199	67.07	89.06
Árbol de Decisión	0.6317	37.75	57.61
Bosque Aleatorio	0.8779	19.45	33.18
XGBoost	0.5385	46.03	64.50

★ Mejor modelo



Mejor modelo: Bosque Aleatorio

R² = **0.8779**

MAE = **19.45 €**

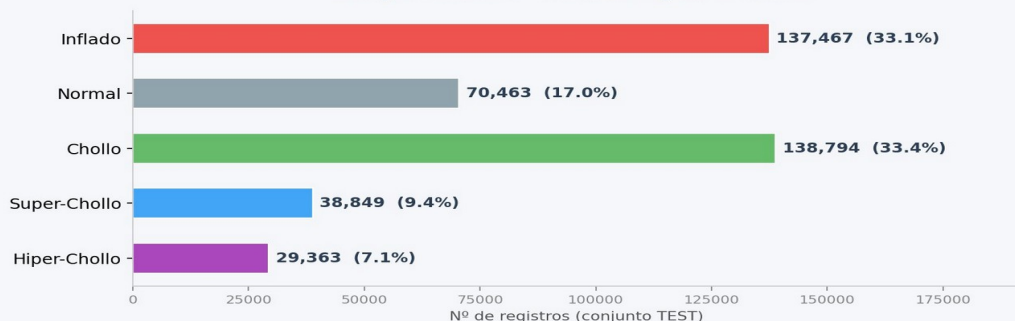
RMSE = **33.18 €**

Explica el 87,8% de la varianza del precio.
Error medio de ±19,45€ sobre el precio real.



Distribución de categorías de chollo — Conjunto TEST (414.936 registros)

Distribución de categorías de chollo — Conjunto de TEST Bosque Aleatorio • 414.936 registros totales



Interpretación:

Chollos detectados

chollo + super + hiper-chollo

= 206.006 registros

= 49,9% del conjunto TEST

El modelo identifica correctamente
casi 1 de cada 2 alojamientos
como oportunidad de ahorro.

Inflado + Normal = 207.930 (50.1%)

Categorización de Chollos · Núcleo del Proyecto

`crear_etiqueta_chollo(y_real, y_predicho) → ratio precio_real / precio_predicho`

4 Hiper-chollo

ratio ≤ 0.75
>25% más barato que el precio justo

3 Super-chollo

$0.75 < \text{ratio} < 0.85$
15-25% más barato

2 Chollo

$0.85 \leq \text{ratio} < 0.97$
3-15% más barato

1 Normal

$0.97 \leq \text{ratio} \leq 1.03$
precio dentro del $\pm 3\%$

0 Inflado

ratio > 1.03
más de un 3% por encima

crear_etiqueta_chollo()

```
def crear_etiqueta_chollo(y_real, y_predicho):
    """
    Compara precio real vs. precio predicho por
    el modelo de regresión. El ratio indica cuánto
    vale el alojamiento respecto a su 'precio justo'.
    """
    ratio = y_real / y_predicho # < 1 = más barato

    condiciones = [
        ratio <= 0.75,                # hiper_chollo
        (ratio > 0.75) & (ratio < 0.85), # super_chollo
        (ratio >= 0.85) & (ratio < 0.97), # chollo
        (ratio >= 0.97) & (ratio <= 1.03), # normal
        ratio > 1.03                  # inflado
    ]
    etiquetas = [4, 3, 2, 1, 0]

    # np.select aplica condiciones vectorizadas
    return pd.Series(
        np.select(condiciones, etiquetas),
        index=y_real.index,
        name='categoria')
```

¿Por qué no ML de clasificación?

La categoría chollo es consecuencia directa de la comparación entre precio real y precio predicho.

Un chollo no tiene sentido sin su precio de referencia.

Entrenar un modelo de clasificación sobre etiquetas derivadas del propio modelo de regresión sería redundante y añadiría complejidad sin valor.

La regla determinista (ratio) es interpretable, reproducible y no introduce sesgo de aprendizaje.

Pipeline completo:



Evaluación del modelo de regresión sobre X_{test} (una sola vez): R^2 , MAE, RMSE · joblib guarda modelo + scalers para producción

Aplicación Web • Streamlit • Arquitectura

App/run.py → streamlit run App/main.py • múltiples módulos especializados

📁 Estructura de la App (TFG_Chollos/App/)

main.py

Página principal: mapa corofético + gráficos de análisis

pages/Busqueda.py

Página de búsqueda: formulario + 16 filtros + resultados

_scraper_app.py

Scraping async (Playwright) + BookingExtractor (ThreadPool)

_feature_engineering.py

preprocesar_nuevos() + codificar_nuevos() con encoders guardados

_predictor.py

cargar_modelos() + predecir_nuevos() + mostrar_resultados()

run.py

Lanzador: subprocess.run([streamlit, run, App/main.py])

🔄 Flujo real (Busqueda.py)

1

Formulario usuario

Provincia/localidad, fechas, tipo, 16 filtros (piscina, spa, mascotas, desayuno...)

2

scrape_busqueda()

Playwright async → listado Booking + BookingExtractor → fichas detalle (max 25 por lugar)

3

preprocesar_nuevos()

Extrae servicios, distancia Haversine, es_finde, dias_restantes... Igual que preprocessing.py

4

codificar_nuevos()

Aplica le_localidad.pkl + le_provincia.pkl + columnas_modelo.pkl guardados en training

5

predecir_nuevos()

bosque_aleatorio_reg.pkl → y_pred × n_noches → crear_etiqueta_chollo() → categoría

6

mostrar_resultados()

st.dataframe() ordenado por ahorro + gráfico pie de categorías con plotly

Conclusiones y Trabajo Futuro

Reflexiones finales sobre el proyecto

1 El precio “justo” es predecible

El Bosque Aleatorio explica el 87,8 % de la varianza del precio ($R^2 = 0.8779$) con un error medio de $\pm 19,45$ €. El precio de un alojamiento no es aleatorio: depende fundamentalmente del tamaño, la ubicación y la temporalidad.

2 Casi 1 de cada 2 alojamientos es un chollo

El 49,9 % de los alojamientos del conjunto de test tienen un precio inferior al predicho. Booking.com tiene una variabilidad de precios real que el usuario no puede percibir sin apoyo de ML.

3 La calidad del dato importa más que el modelo

El scraping en 2 fases, la geocodificación, la extracción de servicios binarios y la gestión de outliers fueron más determinantes para el resultado final que la elección del algoritmo.

4 Un sistema end-to-end es más que la suma de sus partes

Conectar scraping → preprocesamiento → modelización → app web en un único sistema reproducible es el verdadero reto de ingeniería. El modelo solo funciona si todos los eslabones están bien alineados.



Líneas de trabajo futuro

→ Despliegue Streamlit Cloud

→ ML no supervisado

→ Ampliar a Airbnb / Idealista

→ Alertas automáticas

¡Gracias!

Detector de Chollos en Alojamientos Turísticos de Andalucía

Mario Jiménez · Trabajo de Fin de Grado · Universidad de Sevilla · 2025-2026



github.com/Marjimleo3/TFG